

Planificación inicial del trabajo

Proyecto final — Métodos numéricos

Índice

1. Introducción	1
2. Calibración y coste computacional	2
3. Revisión bibliográfica	3
4. Análisis del paper de deep surrogates	6
4.1. ¿Qué proponen en el paper?	6
4.2. Comparación de redes neuronales con otras alternativas	7
4.3. Decisiones clave de diseño	8
4.4. Cómo validar la precisión de los surrogates	9
4.5. Aplicaciones prácticas	10
5. El enfoque de nuestro trabajo	10
6. Decisiones pendientes	12

1. Introducción

El objetivo general de este trabajo es estudiar cómo se pueden usar las redes neuronales como herramienta numérica para aproximar modelos de pricing de opciones. Es decir, no nos interesa la red como un sistema que aprende del mercado, sino como un aproximador funcional capaz de imitar a un solver matemático ya conocido. Esta distinción puede parecer sutil pero cambia completamente lo que se está haciendo: no estamos prediciendo nada, estamos sustituyendo un cálculo costoso por una aproximación rápida y diferenciable.

En finanzas cuantitativas hay muchos modelos de pricing de opciones, desde el clásico Black-Scholes hasta modelos modernos con volatilidad estocástica, saltos o memoria. Todos ellos tienen una estructura común: dado un conjunto de inputs, que pueden ser características del contrato y parámetros del modelo, devuelven un precio o una volatilidad implícita. El problema es que conforme los modelos ganan realismo también se vuelven más caros computacionalmente. Lo que en Black-Scholes es una fórmula cerrada se convierte en Heston en una integral que hay que resolver por Fourier, y en Bates o en rough volatility en algo aún más pesado. Cuando estos modelos hay que evaluarlos miles o millones de veces, por ejemplo en calibración diaria, en cálculo de Greeks, en simulación de escenarios o en análisis out-of-sample, el coste se vuelve un cuello de botella real.

La idea de los deep surrogates es resolver ese cuello de botella entrenando una red neuronal a partir de datos sintéticos generados por el propio modelo, de forma que la red aprenda la función de pricing. Si llamamos $f : \mathcal{X} \subset \mathbb{R}^d \rightarrow \mathbb{R}$ a la función de pricing del modelo verdadero, lo que

hacemos es entrenar una red $\hat{f}_\theta$ con parámetros θ para que aproxime

$$\hat{f}_\theta(x) \approx f(x), \quad x \in \mathcal{X}. \quad (1)$$

Una vez entrenada, evaluarla es órdenes de magnitud más rápido que llamar al solver original, y además los gradientes $\nabla_x \hat{f}_\theta$ se obtienen gratis por diferenciación automática, lo cual es muy útil para Greeks y calibración. La lógica completa se puede resumir en una sola línea:

`modelo financiero → datos sintéticos → red neuronal → evaluación rápida`

La red no reemplaza la teoría financiera, simplemente aprende a imitar al solver. Por eso este planteamiento encaja tan bien con una asignatura de métodos numéricos: lo que estamos haciendo realmente es aproximación de funciones, análisis de error, cálculo de derivadas y eficiencia computacional. No es machine learning aplicado a datos de mercado ruidosos, es un problema clásico de aproximación numérica resuelto con una herramienta moderna.

La frase que mejor captura la motivación del enfoque es que la red neuronal no sustituye al modelo financiero, sino que lo hace operativo a gran escala. En Black-Scholes la fórmula cerrada ya es muy rápida y un surrogate no aporta gran cosa en velocidad, pero precisamente por eso es ideal como caso de validación, porque conocemos la solución exacta y podemos medir errores con precisión. Donde el enfoque empieza a tener sentido real es en modelos donde cada evaluación es costosa, y ahí es donde la metodología tiene un valor industrial claro y muy de actualidad en mesas de derivados y en research cuantitativo.

2. Calibración y coste computacional

Para entender el marco de nuestro proyecto, primero conviene detenerse en qué consiste exactamente el problema de fondo, que es la calibración de modelos de pricing y el coste computacional asociado a ella. Toda la metodología cobra sentido cuando se ve este cuello de botella en detalle, así que merece la pena dedicarle una sección antes de comenzar con la revisión bibliográfica.

Primero es relevante comprender los modelos de pricing de derivados y por qué la calibración es un problema crítico a nivel industrial. Un modelo de pricing financiero es esencialmente una función que toma como entrada la información del mercado y las características de un contrato y devuelve un precio teórico justo. Estos modelos se emplean dentro de las mesas de derivados de bancos de inversión, market makers, hedge funds y son la herramienta de trabajo diaria de quants y traders. Se usan para valorar contratos nuevos cuando un cliente pide cotización, decidir cómo cubrir las posiciones que ya están en cartera, gestionar el riesgo agregado del libro, calcular sensibilidades para reporting regulatorio o marcar a mercado las posiciones al cierre. Sin un modelo bien calibrado, todas estas operaciones pierden el sentido.

Calibrar un modelo es básicamente ajustar sus parámetros internos para que reproduzca precios observados de mercado. Como el mercado ya está cotizando muchas opciones con suficiente liquidez. Una vez calibrado el modelo y ha capturado las dinámicas del mercado en ese momento, se puede usar para valorar opciones exóticas o menos líquidas, calcular sensibilidades para cobertura o simular escenarios. La frecuencia con la que se calibra depende del mercado y del producto, pero en una mesa de derivados de equities o de índices lo habitual es recalibrar al menos una vez al día, normalmente justo antes de la apertura, y en muchos casos varias veces a lo largo del día si el mercado se mueve mucho. La razón es que el smile y la skew de la superficie de volatilidad cambian constantemente, y si los parámetros se quedan desfasados los precios que el modelo devuelve ya no concuerdan con los del mercado, lo que se traduce en mala cobertura, P&L ruidoso y, en última instancia, pérdidas reales. Por eso una mesa de derivados no se permite calibraciones que tarden horas: si el coste computacional es prohibitivo, el modelo simplemente no es operativo.

Para entender bien qué se está calibrando conviene introducir aquí una distinción que vertebrará todo el pricing estructural: la diferencia entre estados y parámetros. Los estados son las variables que describen la situación concreta en la que estás valorando una opción: cambian de un contrato a otro y de un día a otro porque el subyacente se mueve, queda menos tiempo a vencimiento, la volatilidad implícita evoluciona. Los estados típicos en un modelo de pricing son cosas como el precio del subyacente o la moneyness, el tiempo a vencimiento, la varianza instantánea, el tipo de interés y el dividend yield, y responden a la pregunta “¿dónde estamos ahora mismo?”. Los parámetros, en cambio, son las características estructurales del modelo: son lo que define cómo se comporta el mercado.

Formalmente esto se traduce en un problema de optimización inverso. Dado un conjunto de precios reales, buscamos el vector de parámetros que minimiza la suma de errores cuadráticos entre precio de mercado y precio del modelo,

$$\hat{\theta} = \arg \min_{\theta} \sum_{i=1}^{n_{\text{opt}}} \left(C_i^{\text{mercado}} - C_i^{\text{modelo}}(s_i, \theta) \right)^2. \quad (2)$$

Aquí está la trampa: para resolver ese argmin, el optimizador va a llamar al solver del modelo muchas veces. En cada iteración prueba un vector candidato de parámetros, calcula precios para varias decenas de opciones, mide el error, calcula gradientes y actualiza. Una calibración típica puede acabar requiriendo miles de evaluaciones del modelo solo para fijar los parámetros de un único día.

Este número de evaluaciones se puede hacer en segundos o en horas, y aquí es donde el modelo concreto tiene mucha relevancia. En Black-Scholes una evaluación es prácticamente gratis porque tienes fórmula cerrada, así que calibrar es cuestión de segundos. En Heston ya no hay fórmula cerrada pero existe una representación semi-cerrada vía la función característica que se integra por Fourier, así que cada precio cuesta del orden de milisegundos y la calibración entera puede llevar minutos. En modelos que añaden saltos al precio del activo sumas el coste de Heston más los términos de saltos, así que cada precio es algo más lento. De este modo, a medida que se incrementa la complejidad y realismo de los modelos, el coste de las evaluaciones aumenta y las calibraciones se vuelven más lentas y computacionalmente costosas.

En este contexto se enmarca el objetivo de nuestro trabajo y de los surrogates, la idea del enfoque es promover los parámetros a la categoría de pseudo-estados: en vez de entrenar una red para una calibración concreta y volver a entrenarla cuando los parámetros cambien, la red recibe en el input tanto los estados como los parámetros, así que aprende una familia entera de funciones de pricing parametrizadas. Una vez entrenada, evaluarla es órdenes de magnitud más rápido que llamar al solver original, y los gradientes respecto a parámetros y estados se obtienen por diferenciación automática. Esto cambia totalmente la economía de la calibración, el optimizador evita las evaluaciones costosas, los gradientes ya no necesitan diferencias finitas.

3. Revisión bibliográfica

La primera fase de nuestro trabajo ha consistido en realizar una investigación bibliográfica profunda para tener una visión global de los temas de estudio más relevantes. Hay cinco papers que hemos trabajado a fondo y que forman el núcleo de la revisión: los tres primeros eran una recomendación de los profesores como punto de partida, un cuarto, el de deep surrogates, lo encontramos nosotros buscando trabajos más recientes en la línea de aproximación numérica de modelos de pricing, y un quinto, el de differential machine learning de Huge y Savine, lo hemos incorporado como complemento al anterior debido a que el tema de differential machine learning nos parece otra línea de investigación que puede ser interesante para nuestro trabajo.

El punto de partida histórico es el trabajo de Hutchinson, Lo y Poggio de 1994 [4]. Estos autores proponen un método no paramétrico para estimar la fórmula de pricing de un derivado mediante learning networks, en una época en la que la literatura financiera estaba dominada por enfoques paramétricos basados en Black-Scholes y argumentos de no arbitraje. La idea central es invertir la lógica habitual: en lugar de partir de un modelo estocástico para el subyacente y derivar analíticamente la fórmula del precio, dejan que la red aprenda directamente la función que mapea variables económicas observables, como el precio del subyacente normalizado por el strike y el tiempo a vencimiento, al precio de la opción, sin imponer supuestos de lognormalidad ni continuidad de trayectoria. Comparan cuatro arquitecturas, redes de funciones de base radial, perceptrones multicapa, projection pursuit regression y mínimos cuadrados ordinarios como baseline, y atacan a la vez los problemas de pricing y delta-hedging, lo cual es interesante porque la cobertura exige que la red aproxime bien no solo el precio sino también su derivada respecto al subyacente. Validan primero con datos sintéticos generados por Monte Carlo bajo Black-Scholes y después con datos reales de opciones sobre futuros del S&P 500 entre 1987 y 1991, donde la red llega a superar a Black-Scholes en tracking error de la cartera replicante. Su relevancia histórica reside en que es el primer trabajo que demuestra de forma sistemática que una red neuronal puede aprender una fórmula de pricing y usarse operativamente para cubrir, abriendo la línea de investigación que conecta directamente con el enfoque actual de deep surrogates. La diferencia clave es que aquí la red aprende del mercado, mientras que un surrogate moderno aprende la salida de un pricer ya conocido para acelerarlo, lo que cambia el papel de la red de competidor de Black-Scholes a aproximador eficiente del mismo.

El segundo paper recomendado es el de Becker, Cheridito y Jentzen sobre deep optimal stopping, de 2019 [8]. Aquí los autores proponen un método de deep learning para resolver problemas de optimal stopping en tiempo discreto, es decir, problemas en los que hay que decidir en cada instante si parar o continuar un proceso de Markov para maximizar la esperanza de un pago. La idea central es que cualquier tiempo de parada admisible se puede descomponer en una sucesión de decisiones binarias, una por cada paso temporal, y que cada una de esas decisiones puede aprenderse como una función del estado actual mediante una red neuronal feedforward. Sustituyen la indicadora dura por una sigmoide, lo que permite entrenar los parámetros con ascenso de gradiente estocástico sobre trayectorias Monte Carlo, y proceden por inducción hacia atrás desde el último instante hasta el primero. El método se aplica al pricing de opciones bermudas tipo max-call sobre cestas de hasta quinientos activos en un Black-Scholes multidimensional, a un callable multi barrier reverse convertible y al problema no markoviano de parar óptimamente un movimiento browniano fraccionario, con precisión alta y tiempos cortos en dimensiones donde los métodos tradicionales sufren la maldición de la dimensionalidad. Para nuestra revisión es importante porque ilustra una línea claramente distinta de la nuestra: ellos aprenden directamente la política de ejercicio para derivados con derecho de parada anticipada, mientras que en nuestro proyecto la red actuará como surrogate de la función de pricing de opciones europeas, aproximando el mapa de parámetros del modelo al precio sin necesidad de modelar decisiones de ejercicio. Son enfoques complementarios dentro del uso del deep learning en finanzas cuantitativas, pero no resuelven el mismo problema.

El tercer paper recomendado es la revisión bibliográfica de Ruf y Wang de 2020 [11], que funciona como mapa general del campo. Los autores compilan y comparan más de ciento cincuenta artículos en una tabla detallada que cataloga cada estudio según seis dimensiones, entre ellas las variables de entrada, las salidas de la red, los modelos benchmark, las métricas de rendimiento, el método de partición de los datos y los activos subyacentes. Cubren tanto el enfoque puramente no paramétrico, donde la red aprende directamente el precio a partir de variables como subyacente, strike, vencimiento y volatilidad, como los enfoques híbridos que combinan fórmulas paramétricas tipo Black-Scholes con correcciones aprendidas, además de variantes más recientes orientadas a calibración, resolución de PDEs, aproximación de funciones de valor en problemas de control

óptimo y deep hedging. Buena parte de su valor reside en que identifican errores metodológicos recurrentes en la literatura: la importancia de usar inputs estacionarios como la moneyness en lugar de precio y strike por separado, la necesidad de elegir benchmarks que no trivialicen la comparación, y sobre todo el problema del data leakage que aparece cuando la partición train/test se hace de manera aleatoria sobre datos con estructura temporal, rompiendo el orden cronológico, contaminando el test e infraestimando sistemáticamente el error de generalización. Para nuestro trabajo es una referencia muy útil porque ofrece un panorama de la disciplina, permite situar nuestra propuesta dentro de una tradición ya consolidada y nos advierte de errores típicos que conviene evitar al diseñar inputs, elegir benchmarks y construir las particiones de datos.

El cuarto paper, y el que de hecho nos ha terminado convenciendo como referencia central, es el de Chen, Didisheim y Scheidegger de 2025 sobre deep surrogates para finanzas [14]. Como ya hemos descrito en detalle en las secciones anteriores, propone los deep surrogates como una clase de aproximadores numéricos para modelos estructurales costosos, sustituyendo la función original por una red neuronal profunda entrenada con muestras simuladas que toma exactamente los mismos inputs y devuelve la misma salida a un coste computacional drásticamente menor, además de proporcionar gradientes gratis vía diferenciación automática. La idea descansa en que, gracias al teorema de aproximación universal y a resultados recientes sobre modelos afines, una red profunda puede romper la maldición de la dimensionalidad creciendo solo polinómicamente con la dimensión del input. Lo aplican a una versión extendida del modelo de Bates con saltos doble exponenciales asimétricos, en un espacio aumentado de dimensión trece, mapean los precios a Black-Scholes implied volatility para que los errores sean comparables entre opciones de distinta moneyness y vencimiento, eligen Swish frente a ReLU para que las Deltas sean estables, validan por bins de moneyness y madurez, y demuestran que el surrogate reduce la calibración diaria del modelo de las ciento veinticinco horas que necesita la inversión FFT a poco más de dos horas en CPU y unas decenas de minutos en GPU. Esa reducción de tiempos no es un detalle, es lo que abre la puerta a aplicaciones empíricas que antes eran inviables, como el índice diario TailRisk para predecir crashes, la medida de inestabilidad de parámetros vinculada a la liquidez de mercado, la comparación fuera de muestra contra random forests sobre la superficie de volatilidad implícita y la construcción de distribuciones condicionales de retornos vía VAR y bootstrap.

El quinto paper que hemos terminado promoviendo a este núcleo principal es el de Hugel y Savine de 2020 sobre differential machine learning [10]. Originalmente lo teníamos como uno más del panorama, pero al pensar en cómo podríamos innovar respecto al paper de deep surrogates nos hemos dado cuenta de que ofrece una pieza metodológica muy potente y muy alineada con nuestras preocupaciones. La idea es sencilla y elegante. Cuando entrenas un surrogate con datos sintéticos, normalmente la red solo ve el valor de la función en cada punto, por ejemplo el precio de la opción, y aprende a predecirlo. Sin embargo, si el modelo verdadero es diferenciable, y en finanzas es lo habitual, también conoces gratis la pendiente de la función en cada punto, es decir, las Deltas, Vegas y demás derivadas. Differential machine learning aprovecha esa información añadiendo los gradientes verdaderos al conjunto de entrenamiento y modificando la pérdida para que la red no solo acierte el precio, sino también su derivada respecto a los inputs. El resultado es que se consiguen aproximaciones mucho más precisas, especialmente en términos de Greeks, con datasets sustancialmente más pequeños. Para nuestro trabajo es una referencia clave porque conecta directamente con la necesidad de validar Greeks que ya teníamos heredada del paper de Chen y porque tiene una motivación matemática desde el punto de vista de aproximación de funciones.

Más allá de estos cinco pilares, hemos echado un vistazo a otros papers que ayudan a entender que la aplicación de redes neuronales a derivados ha avanzado en varias direcciones complementarias. Una primera línea ataca de frente el problema de resolver las ecuaciones diferenciales parciales que aparecen en el pricing de derivados de alta dimensión, donde los métodos de mallado tradicionales chocan con la maldición de la dimensionalidad: en esa familia se inscribe el Deep Galerkin

Method de Sirignano y Spiliopoulos de 2018 [5], que entrena una red profunda para satisfacer el operador diferencial sobre puntos muestreados al azar y prescinde por completo de la malla, así como el Deep BSDE Solver de Han, Jentzen y E del mismo año [6], que reformula la PDE como una ecuación diferencial estocástica retrógrada y aproxima el gradiente de la solución mediante redes neuronales, demostrando viabilidad incluso en problemas de cien dimensiones. Una segunda línea busca acelerar la valoración aprendiendo directamente la función de precios, y aquí el trabajo de Ferguson y Green de 2018 [7] muestra que una red profunda entrenada con datos sintéticos generados por Monte Carlo puede valorar opciones hasta millones de veces más rápido que el modelo subyacente. Una tercera vertiente aborda la cobertura de carteras como un problema de aprendizaje por refuerzo, como en el deep hedging de Buehler, Gonon, Teichmann y Wood de 2019 [9], que parametriza la estrategia de cobertura mediante redes neuronales y la optimiza directamente bajo medidas de riesgo convexas, incorporando de forma natural costes de transacción y otras fricciones de mercado sin necesidad de griegas. La calibración de modelos estocásticos también se ha beneficiado de estas técnicas, como ilustra el paper de calibración de volatilidad estocástica de 2023 [13] al aplicar differential ML al modelo de Heston para reducir drásticamente el tiempo de calibración manteniendo la precisión, y finalmente Della Corte y coautores en 2023 [12] cierran el cuadro con un estudio empírico sistemático que muestra que arquitecturas tipo highway o variantes de DGM superan al perceptrón multicapa estándar en tareas de pricing y volatilidad implícita, sugiriendo que el diseño de la red importa tanto como el problema financiero al que se aplica.

Después de revisar todo este material, la conclusión a la que hemos llegado es que el paper de deep surrogates es el que mejor encaja con . La razón principal es que el enfoque de surrogates aborda un problema que en la práctica de la industria es muy importante, que es mejorar el coste computacional de los modelos de pricing y reducir los tiempos de calibración y estimación de parámetros. Esos son temas que están muy de actualidad en mesas de derivados y en research cuantitativo, sobre todo a medida que los modelos se vuelven más realistas y por tanto más caros de evaluar. Y desde el punto de vista de métodos numéricos nos parece más interesante. Con los surrogates estamos haciendo aproximación de funciones en alta dimensión, controlando errores por regiones del dominio, calculando derivadas por diferenciación automática y discutiendo ganancias de eficiencia computacional, que es exactamente lo que cabe esperar de un trabajo de la asignatura. Por eso, aunque los demás papers nos han servido para entender el panorama, el de Chen, Didisheim y Scheidegger es el que vamos a usar como ancla del proyecto.

4. Análisis del paper de deep surrogates

Como el paper de Chen, Didisheim y Scheidegger es la referencia central del trabajo, conviene dedicarle un análisis detallado. En las siguientes subsecciones repasamos primero qué proponen y a qué modelo lo aplican, después por qué eligen redes neuronales frente a otras alternativas de aproximación numérica, las decisiones técnicas concretas que toman al construir el surrogate, cómo validan que funciona, y por último las aplicaciones empíricas que se abren gracias a tener un surrogate operativo.

4.1. ¿Qué proponen en el paper?

El paper aplica todo esto al modelo de Bates [3], este modelo combina volatilidad estocástica al estilo Heston con saltos en el precio del activo, donde los saltos siguen una distribución doble exponencial asimétrica en lugar de la normal del Bates original. La dinámica del subyacente y la

varianza bajo la medida de riesgo neutral se puede escribir como

$$\frac{dS_t}{S_{t-}} = (r - q - \lambda \bar{\nu}) dt + \sqrt{v_t} dW_t^S + d\left(\sum_{i=1}^{N_t} (e^{J_i} - 1)\right), \quad (3)$$

$$dv_t = \kappa(\theta - v_t) dt + \xi \sqrt{v_t} dW_t^v, \quad dW_t^S dW_t^v = \rho dt, \quad (4)$$

donde N_t es un Poisson de intensidad λ y los tamaños de salto J_i siguen una doble exponencial asimétrica

$$f_J(z) = p \eta_+ e^{-\eta_+ z} \mathbf{1}_{\{z \geq 0\}} + (1 - p) \eta_- e^{\eta_- z} \mathbf{1}_{\{z < 0\}}. \quad (5)$$

Esto les permite capturar cosas que Black-Scholes no puede capturar bien, como el leverage effect, que es el fenómeno por el cual cuando el mercado cae la volatilidad suele subir (capturado por $\rho < 0$), o la asimetría entre saltos positivos y negativos, que es importante porque en realidad las caídas extremas y las subidas extremas no se comportan igual. La distribución doble exponencial permite modelar colas más pesadas que la normal y separar el comportamiento de saltos hacia arriba y hacia abajo.

Cuando llevan Bates al framework del surrogate, acaban con un input de trece dimensiones. Tienen cinco variables de estado, que son la moneyness normalizada, el tiempo a vencimiento, la varianza instantánea, el tipo de interés y el dividend yield, y ocho parámetros estructurales, que controlan la velocidad de reversión a la media de la volatilidad, la varianza de largo plazo, la volatilidad de la varianza, la correlación entre precio y volatilidad, la intensidad de saltos, los tamaños medios de saltos positivos y negativos y la probabilidad de que un salto sea positivo. Formalmente, el input de la red es

$$x = \underbrace{(m, \tau, v_0, r, q)}_{5 \text{ estados}} \cup \underbrace{(\kappa, \theta, \xi, \rho, \lambda, \eta_+, \eta_-, p)}_{8 \text{ parámetros}} \in \mathbb{R}^{13}, \quad (6)$$

y la red aprende una función $\hat{f}_\theta(x)$ que devuelve la volatilidad implícita Black-Scholes generada por Bates.

Aquí hay algo importante que conviene tener muy presente, y es que el tamaño final de la muestra de entrenamiento que usan es de mil millones de observaciones, $N = 10^9$. Aunque parece una barbaridad, los autores recalcan que en un espacio de dimensión trece eso sigue siendo una muestra muy dispersa. Si hicieras una malla cartesiana con solo diez valores por dimensión, necesitarías 10^{13} puntos, que es mucho más. Esa observación es relevante porque demuestra que la red no está memorizando una tabla densa, sino que realmente está aprendiendo una estructura funcional. Y aquí entra una de las justificaciones teóricas más fuertes del enfoque: para ciertos tipos de modelos financieros, en particular los modelos afines como Bates, hay resultados teóricos que muestran que el número de parámetros entrenables y el tamaño de la muestra necesarios para aproximar bien la solución crecen polinómicamente con la dimensión del input, no exponencialmente. Eso ayuda a aliviar la maldición de la dimensionalidad y es justo la razón por la que las redes neuronales son una herramienta interesante en este contexto.

4.2. Comparación de redes neuronales con otras alternativas

El paper compara explícitamente las redes neuronales profundas con otros métodos de aproximación que serían candidatos naturales, como polinomios, splines, sparse grids y Gaussian processes. La comparación es bastante clarificadora. Los polinomios funcionan razonablemente en funciones suaves pero les cuesta capturar rasgos locales fuertes como kinks, y eso en opciones es un problema serio porque el payoff tiene un kink claro en el strike. Si intentas aproximar eso con polinomios globales aparecen oscilaciones no deseadas, que es el famoso fenómeno de Runge. Los splines capturan bien rasgos locales porque trabajan por tramos, pero escalan muy mal en alta dimensión. Los sparse grids resuelven parcialmente el problema de las mallas cartesianas,

pero funcionan peor en dominios irregulares, y los modelos financieros muchas veces tienen restricciones entre parámetros que generan dominios complicados. Los Gaussian processes son potentes y elegantes, pero escalan fatal con muchos datos porque las operaciones matriciales son del orden de n al cubo, así que con millones de observaciones se vuelven inviables. Las redes neuronales cumplen los cuatro criterios de forma razonable: funcionan en alta dimensión, capturan no linealidades, admiten dominios irregulares y escalan bien con muchos datos.

Hay además otra ventaja muy importante de las redes que el paper subraya y que conecta directamente con el espíritu de métodos numéricos, que es que el Jacobiano de la red, $\partial \hat{f}_\theta / \partial x$, está disponible casi gratis mediante diferenciación automática o backpropagation. Esto es crítico en finanzas porque muchas magnitudes que importan no son el precio en sí sino sus derivadas:

$$\Delta = \frac{\partial C}{\partial S}, \quad \mathcal{V} = \frac{\partial C}{\partial \sigma}, \quad \rho = \frac{\partial C}{\partial r}, \quad (7)$$

todas ellas fundamentales para cobertura y gestión de riesgo. Si trabajas con el modelo original como una caja negra muchas veces tienes que calcular derivadas mediante diferencias finitas,

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + h e_i) - f(x - h e_i)}{2h}, \quad (8)$$

lo que multiplica el coste computacional por el número de inputs. Con una red neuronal las derivadas salen prácticamente gratis al pasar gradientes hacia atrás. Y lo mismo ocurre cuando quieres estimar parámetros con GMM o calibrar el modelo: si la función objetivo se evalúa rápido y sus gradientes se obtienen automáticamente, la calibración se vuelve muchísimo más barata. De hecho el paper documenta que estimar los parámetros con el método tradicional vía FFT lleva alrededor de ciento veinticinco horas, mientras que con el surrogate lleva poco más de dos horas en CPU y unas decenas de minutos en GPU. Es decir, no es una mejora marginal, es una reducción de órdenes de magnitud.

4.3. Decisiones clave de diseño

Hay cuatro decisiones técnicas en el paper que nos parecen especialmente importantes y que queremos incorporar a nuestro propio enfoque. Como son discretas y conviene tenerlas separadas para no perderlas de vista cuando luego implementemos, en este caso sí merece la pena enumerarlas:

- **Activación suave en lugar de ReLU.** Intuitivamente uno pensaría que ReLU, definida como $\text{ReLU}(x) = \max(x, 0)$, es la elección natural porque se parece al payoff de una call $(S - K)^+$, con esa forma de cero hasta cierto punto y luego crecimiento lineal. Sin embargo los autores eligen Swish, que se puede ver como una versión suave de ReLU:

$$\text{Swish}(x) = x \cdot \text{sigmoid}(\beta x) = \frac{x}{1 + e^{-\beta x}}. \quad (9)$$

La razón es que ReLU no es diferenciable en el cero y, aunque eso puede no afectar mucho al precio aproximado, sí afecta a las derivadas que la red produce. Hacen un experimento muy revelador donde entrenan dos surrogates idénticos salvo por la activación y encuentran que los errores de pricing son comparables pero los errores de Delta son sustancialmente mayores con ReLU. La lección que queremos heredar es que para un surrogate financiero no basta con mirar el error en precio, también hay que mirar la calidad de las derivadas, y la suavidad de la activación importa más para los Greeks que para el precio.

- **Volatilidad implícita como métrica de evaluación.** El paper convierte los precios a volatilidad implícita Black-Scholes y mide errores en esa escala. La volatilidad implícita σ_{IV} asociada a un precio observado C^* se define implícitamente como la solución de

$$C_{BS}(S, K, T, r, \sigma_{IV}) = C^*, \quad (10)$$

es decir, la σ que habría que meter en Black-Scholes para reproducir ese precio. Comparar errores en precio puede ser muy engañoso porque un error de veinte céntimos no significa lo mismo en una opción que vale medio euro que en una que vale veinticinco, ni en una corta que en una larga, ni en una in-the-money que en una out-of-the-money. La volatilidad implícita actúa como unidad común que pone todas las opciones en una escala más homogénea. Aunque el modelo verdadero sea Bates o Heston, se toma el precio que genera y se busca qué volatilidad habría que meter en Black-Scholes para producir ese mismo precio, y luego se comparan volatilidades implícitas en lugar de precios. Esto es estándar en la práctica del mercado de opciones y es una decisión metodológica que deberíamos copiar tal cual.

- **Generación de datos sobre un hipercubo con muestreo flexible.** La lógica es directa: se define el espacio de inputs como un hipercubo

$$\mathcal{X} = \prod_{i=1}^d [a_i, b_i], \quad (11)$$

fijando mínimo y máximo para cada variable, se generan puntos aleatorios $\{x^{(j)}\}_{j=1}^N \sim \mathcal{X}$ dentro y para cada punto se evalúa el modelo verdadero $y^{(j)} = f(x^{(j)})$, formando así la muestra sintética $\mathcal{D} = \{(x^{(j)}, y^{(j)})\}_{j=1}^N$. La distribución de muestreo no tiene por qué ser uniforme: puede ser uniforme simple, basada en priors jerárquicos, o concentrarse en zonas donde la función es más no lineal o donde la red está cometiendo más error, lo cual conecta con active learning y es algo que nos gustaría explorar como elemento diferencial. Hay también un detalle técnico importante, y es que las redes suelen aproximar peor cerca de los bordes del dominio, así que los autores generan los datos en un rango ligeramente reducido, quitando un cinco por ciento por cada lado, para evitar que la evaluación quede dominada por errores de frontera.

- **Lógica de entrenamiento distinta a la del ML clásico.** En machine learning normal se suele usar early stopping, dropout y otras técnicas de regularización para evitar overfitting. El entrenamiento estándar consiste en minimizar una pérdida empírica

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{j=1}^N \ell(\hat{f}_{\theta}(x^{(j)}), y^{(j)}), \quad (12)$$

y en un surrogate eso no es tan necesario porque los datos son sintéticos y casi sin ruido, ya que el target viene del propio modelo, y si la red empieza a fallar en alguna zona siempre se pueden generar más datos. No estamos intentando aprender patrones inciertos a partir de datos limitados y ruidosos, sino aproximando una función determinista conocida pero cara de evaluar. Esa diferencia conceptual es la que justifica que los hiperparámetros y la lógica de entrenamiento sean distintos a los de un problema típico de ML.

4.4. Cómo validar la precisión de los surrogates

La validación que hacen en el paper también es interesante y está separada en dos niveles. En el primer nivel suponen que los parámetros verdaderos son conocidos y comparan precio del modelo Bates resuelto con FFT contra precio del surrogate. Para hacerlo de forma seria dividen las opciones en doce grupos según vencimiento y moneyness, con cuatro categorías de tiempo a vencimiento y tres niveles de moneyness ($4 \times 3 = 12$ bins), y dentro de cada grupo generan veinte mil opciones aleatorias. Esa estructura por bins permite ver si el surrogate funciona bien en distintas zonas del mercado, no solo en promedio. Sobre cada bin reportan el error absoluto medio y los percentiles de la distribución del error,

$$\text{MAE}_{\text{bin}} = \frac{1}{N_{\text{bin}}} \sum_{j \in \text{bin}} |\hat{f}_{\theta}(x^{(j)}) - f(x^{(j)})|. \quad (13)$$

Los resultados son muy buenos: el percentil noventa y cinco del error absoluto en volatilidad implícita y en Delta está por debajo de 0,0008 para opciones con más de siete días a vencimiento. Los errores son algo mayores en opciones muy cortas, lo cual tiene sentido porque ahí el payoff está más cerca, hay más no linealidad y las sensibilidades son más bruscas, pero incluso ahí los errores siguen siendo pequeños en términos relativos.

En el segundo nivel de validación van más allá y estudian si los errores de aproximación se traducen en errores de estimación de parámetros. La idea es que en la realidad nadie te da los parámetros verdaderos, hay que estimarlos, y eso introduce una nueva fuente de error. Hacen un experimento de simulación donde fijan parámetros verdaderos, simulan precios, usan GMM con el surrogate para estimar los parámetros, y luego miden el error de pricing fuera de muestra usando esos parámetros estimados frente a los verdaderos, todo evaluado con el solver original para aislar el efecto de la estimación. Encuentran que el impacto económico es pequeño en general. Hay un detalle muy interesante que conviene retener, que es que algunos parámetros pueden estar débilmente identificados, en el sentido de que los precios no son muy sensibles a ellos. En esos casos el error de estimación del parámetro puede parecer grande pero su impacto sobre los precios es pequeño porque la sensibilidad también es pequeña.

4.5. Aplicaciones prácticas

Una parte del paper que se aleja de nuestro alcance pero que conviene entender es cómo el surrogate abre la puerta a aplicaciones empíricas que antes eran demasiado costosas. Como el surrogate permite reestimar Bates diariamente, los autores construyen un índice llamado TailRisk que mide la pérdida esperada bajo medida neutral al riesgo asociada a un salto negativo del índice durante la próxima semana. La idea es que las opciones contienen información sobre cuánto miedo a caídas extremas hay descontado en los precios, y los parámetros del modelo permiten cuantificar eso. Después comprueban si TailRisk predice eventos extremos reales del S&P 500 mediante regresión logística y encuentran que el pseudo R cuadrado casi se duplica frente a una medida no paramétrica anterior basada en opciones.

Otra aplicación que hacen es estudiar la inestabilidad de los parámetros estimados. La intuición es que si los parámetros del modelo cambian mucho día a día, los market makers tienen más dificultad para cubrir sus inventarios, asumen más riesgo y exigen mayor compensación, y eso se acaba reflejando en bid-ask spreads más altos. Encuentran una relación positiva y estadísticamente significativa entre inestabilidad de parámetros y bid-ask spreads relativos en opciones SPX, incluso controlando por características del contrato, volumen anormal, vencimiento y moneyness. Esto sugiere que la inestabilidad del modelo no es solo un problema técnico, tiene consecuencias económicas reales sobre la liquidez del mercado.

Hacen también una comparación muy interesante entre el modelo estructural Bates y un benchmark reduced-form basado en random forests que modela directamente la superficie de volatilidad implícita. Los resultados son matizados. Bates lo hace peor que random forest en opciones muy cortas, de siete días o menos, pero mejor en opciones largas, de más de noventa días a vencimiento. Bates también funciona relativamente mejor en opciones near-the-money y con bid-ask spreads bajos. La interpretación es que las opciones cortas, OTM o con spreads altos son más sensibles a factores de liquidez, demanda puntual y microestructura que Bates no modela bien, mientras que las opciones largas reflejan más riesgos fundamentales donde un modelo estructural aporta más disciplina económica. La conclusión general es que los modelos reduced-form tienden a ganar cuando el modelo estructural está mal especificado, pero generalizan peor fuera del dominio de entrenamiento. Por último usan el surrogate para construir distribuciones condicionales de retornos de opciones mediante un VAR sobre las variables de estado y bootstrapping, y muestran que Bates supera claramente a Heston en la capacidad de capturar las colas de la distribución, lo cual tiene sentido porque Heston no tiene saltos y las colas dependen mucho de los saltos.

5. El enfoque de nuestro trabajo

Después de digerir todo esto, la decisión más importante es cómo enfocar nuestro trabajo para complementar y aportar valor a la bibliografía ya existente, dentro de nuestras limitaciones temporales y computacionales.

La estructura que más sentido nos hace es trabajar en dos niveles. El primer nivel es Black-Scholes [1] como caso de validación. La fórmula cerrada para una call europea es

$$C_{BS}(S, K, T, r, \sigma) = S \Phi(d_1) - K e^{-rT} \Phi(d_2), \quad (14)$$

con

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}, \quad (15)$$

y la Delta analítica $\Delta_{BS} = \Phi(d_1)$. Black-Scholes en la práctica no necesita un surrogate porque la fórmula cerrada ya es muy rápida, pero precisamente por eso es ideal como entorno de prueba. Conocemos la solución exacta, conocemos las Deltas analíticas, podemos comparar precio, volatilidad implícita y Greeks de forma directa, y podemos validar todo el pipeline antes de complicarlo. El segundo nivel es extender a Heston [2], cuya dinámica

$$\frac{dS_t}{S_t} = r dt + \sqrt{v_t} dW_t^S, \quad (16)$$

$$dv_t = \kappa(\theta - v_t) dt + \xi \sqrt{v_t} dW_t^v, \quad dW_t^S dW_t^v = \rho dt, \quad (17)$$

introduce volatilidad estocástica, es bastante más realista y su evaluación requiere métodos numéricos como Fourier semi-cerrado, así que la motivación de tener un surrogate empieza a cobrar sentido real. Es un paso intermedio entre Black-Scholes y Bates que conserva la esencia del paper sin meternos en el infierno de implementar saltos doble exponenciales.

La pregunta de investigación que va a guiar todo esto la formulamos así:

Bajo qué condiciones una red neuronal profunda puede actuar como un surrogate preciso, rápido y diferenciable para funciones de pricing de opciones.

De ahí salen varias subpreguntas naturales que son las que vamos a intentar responder, y como son cinco preguntas discretas conviene listarlas para que queden claras desde el principio:

- Cómo varía el error según moneyness y vencimiento.
- Si es más informativo evaluar el error en precio o en volatilidad implícita.
- Si la elección de una activación suave mejora la calidad de los Greeks frente a una no suave como ReLU.
- Qué ganancia computacional ofrece el surrogate frente al solver original.
- Si una estrategia de muestreo enfocado mejora la precisión en regiones difíciles.

Estas preguntas convierten el trabajo en algo más interesante que una mera implementación, lo convierten en un análisis metodológico sobre cómo se diseña y se valida un surrogate financiero.

Hemos identificado cinco elementos diferenciales que no son simplemente repetir el paper y que pueden aportar valor propio. El primero es comparar de forma sistemática el error en precio frente al error en volatilidad implícita. El paper hace esto, pero nosotros podemos plantearlo como una pregunta metodológica explícita: hasta qué punto puede un surrogate tener un error en precio aparentemente bajo pero un error relevante en implied volatility en ciertas zonas del

dominio. Esto acerca el trabajo a la práctica real del mercado, donde la volatilidad implícita es la unidad natural de comparación.

El segundo elemento diferencial es comparar funciones de activación con foco en la calidad de los Greeks. Vamos a entrenar redes con la misma arquitectura cambiando solo la activación, probando ReLU, Softplus, SiLU/Swish y quizá tanh, y vamos a comparar el MAE de precio, el MAE de Delta y el MAE de volatilidad implícita en cada caso. La hipótesis, alineada con el paper, es que las activaciones suaves producen derivadas más estables aunque los errores de precio sean parecidos. Mostrar visualmente que dos redes con MAE de precio similar pueden tener Deltas muy distintas es una conclusión muy potente para un trabajo de métodos numéricos.

El tercer elemento es comparar muestreo uniforme frente a muestreo enfocado. Vamos a generar dos datasets, uno con muestreo uniforme en todo el dominio y otro con más densidad cerca del at-the-money y vencimientos cortos, que es donde sabemos por adelantado que la función de pricing tiene más curvatura. Después evaluamos ambos surrogates con la misma división por bins y vemos si el muestreo enfocado reduce el error en regiones críticas. Esto conecta directamente con ideas clásicas de métodos numéricos como la elección adaptativa de puntos de malla y la concentración del esfuerzo computacional en zonas difíciles.

El cuarto elemento es la medición seria de eficiencia computacional. En Black-Scholes la ganancia probablemente sea pequeña porque la fórmula cerrada ya es rapidísima, pero el experimento sirve para validar el pipeline. En Heston, si llegamos, la comparación se vuelve mucho más interesante. Vamos a medir el tiempo necesario para valorar grandes cantidades de opciones con el solver original frente al surrogate, y a discutir el trade-off entre el coste inicial de generar datos y entrenar la red y el coste recurrente de evaluación. Esto permite contar bien el argumento del paper, que es que el surrogate compensa cuando el modelo se evalúa muchas veces.

El quinto elemento, que solo abordaremos si vamos bien de tiempo después de cerrar los cuatro anteriores, es estudiar el efecto de la función de pérdida en la calidad del surrogate, con especial atención al enfoque de differential machine learning de Hugué y Savine. Lo planteamos como opcional porque, aunque es posiblemente el experimento más interesante desde el punto de vista metodológico, también es el que más coste añadido tiene: requiere implementar el cálculo de Deltas verdaderas durante la generación de datos, modificar la lógica de entrenamiento para que la pérdida combine error de precio y error de derivada, y diseñar comparaciones cuidadosas para que las conclusiones sean limpias. Si los cuatro elementos diferenciales anteriores avanzan rápido y nos queda margen, este sería el experimento estrella; si no, lo dejamos como línea futura natural. La idea sería encadenar dos pruebas. La primera es comparar pérdidas clásicas manteniendo todo lo demás fijo, para ver cómo cambia la distribución del error por regiones:

$$\mathcal{L}_1(\theta) = \frac{1}{N} \sum_j |\hat{f}_\theta(x^{(j)}) - y^{(j)}|, \quad (18)$$

$$\mathcal{L}_2(\theta) = \frac{1}{N} \sum_j (\hat{f}_\theta(x^{(j)}) - y^{(j)})^2, \quad (19)$$

$$\mathcal{L}_{\text{Huber}}(\theta) = \frac{1}{N} \sum_j \text{Huber}_\delta(\hat{f}_\theta(x^{(j)}) - y^{(j)}). \quad (20)$$

La hipótesis razonable, basada en lo que dice el paper de Chen sobre por qué ellos eligen ℓ_1 , es que ℓ_2 dará mejor MAE en el centro del dominio pero peor en los bordes, y al revés con ℓ_1 , mientras que Huber debería comportarse como un compromiso. La segunda, mucho más interesante, es probar differential ML en serio: en lugar de entrenar la red solo con precios, modificamos la pérdida para que también penalice la diferencia entre la Delta verdadera y la Delta que sale por autograd de la red,

$$\mathcal{L}_{\text{diff}}(\theta) = \alpha \cdot \underbrace{\frac{1}{N} \sum_j |\hat{f}_\theta(x^{(j)}) - y^{(j)}|}_{\text{error de precio}} + \beta \cdot \underbrace{\frac{1}{N} \sum_j \|\nabla_x \hat{f}_\theta(x^{(j)}) - \nabla_x f(x^{(j)})\|}_{\text{error de derivadas}}. \quad (21)$$

La pérdida queda como una combinación ponderada del error de precio y del error de derivada, y la red aprende simultáneamente a acertar el nivel y la pendiente de la función. Lo bonito de esto es que tiene una motivación matemática limpiísima desde el punto de vista de aproximación de funciones, porque es la misma idea que está detrás de la interpolación de Hermite frente a la interpolación clásica: si conoces el valor y la derivada de una función en muchos puntos, puedes reconstruirla con menos puntos. La hipótesis es que con datasets pequeños differential ML domina claramente y que conforme creces el dataset las tres versiones convergen, lo que daría una conclusión muy buena sobre cuándo merece la pena pagar el coste extra de calcular gradientes verdaderos durante la generación de datos. Además, este experimento conecta directamente con la obsesión por validar Greeks que ya teníamos heredada del paper de Chen y le da un respaldo bibliográfico fuerte gracias al paper de Huge-Savine, que ya tenemos en el núcleo de la revisión.

6. Decisiones pendientes

Hay varias cosas que quedan abiertas y que vamos a ir decidiendo a medida que avancemos. Como son decisiones independientes entre sí y conviene tenerlas a mano cuando empecemos a implementar, las recogemos aquí en formato lista:

- **Si metemos un tercer modelo más ambicioso por encima de Heston.** Black-Scholes y Heston están confirmados como los dos modelos base del trabajo: el primero como caso de validación con solución analítica y el segundo como extensión natural donde el surrogate empieza a tener sentido real. La pregunta abierta es si vamos un paso más allá y atacamos un modelo más caro y más actual. Los candidatos serían Stochastic Local Volatility, que combina volatilidad local y estocástica y es el más usado en mesas de derivados, los modelos de rough volatility tipo rough Bergomi o rough Heston, que capturan mejor los smiles de corto vencimiento mediante procesos con memoria pero son no Markovianos y por tanto muy caros, o Bates con saltos, que es la línea directa del paper de Chen. Implementar uno de estos tres requeriría bastante esfuerzo extra, sobre todo en la parte de generar el solver verdadero para producir los datos sintéticos, así que la decisión depende de cómo vayamos de tiempo después de cerrar Heston. Como mínimo los mencionaremos en la memoria como motivación de futuro y diremos que la metodología está diseñada pensando en ellos, aunque no los implementemos.
- **Output de la red: precio o volatilidad implícita.** En Black-Scholes no tiene sentido entrenar la red para que prediga volatilidad implícita si la propia sigma ya es input, porque la implied vol coincide trivialmente con esa sigma. Así que en Black-Scholes vamos a entrenar la red para predecir precio y luego evaluar el error transformado a volatilidad implícita. En Heston, en cambio, sí puede tener sentido entrenar dos versiones, una que prediga precio y otra que prediga directamente la volatilidad implícita asociada al precio de Heston, y comparar ambos enfoques. Esa comparación podría ser otra contribución metodológica relevante.
- **Normalización de inputs.** Las redes entrenan mejor con inputs escalados, y en finanzas es habitual usar variables financieramente normalizadas en lugar de las brutas. En vez de meter S y K por separado, podemos trabajar con moneyness o log-moneyness, lo que reduce la dimensionalidad efectiva del problema y suele estabilizar el entrenamiento. Vamos a probar ambas opciones al principio para ver cuál se comporta mejor.

Referencias

- [1] Black, F. y Scholes, M. (1973). *The Pricing of Options and Corporate Liabilities*. Journal of Political Economy, **81**(3), 637–654.

- [2] Heston, S. L. (1993). *A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options*. The Review of Financial Studies, **6**(2), 327–343.
- [3] Bates, D. S. (1996). *Jumps and Stochastic Volatility: Exchange Rate Processes Implicit in Deutsche Mark Options*. The Review of Financial Studies, **9**(1), 69–107.
- [4] Hutchinson, J. M., Lo, A. W. y Poggio, T. (1994). *A Nonparametric Approach to Pricing and Hedging Derivative Securities via Learning Networks*. The Journal of Finance, **49**(3), 851–889.
- [5] Sirignano, J. y Spiliopoulos, K. (2018). *DGM: A Deep Learning Algorithm for Solving Partial Differential Equations*. Journal of Computational Physics, **375**, 1339–1364.
- [6] Han, J., Jentzen, A. y E, W. (2018). *Solving High-Dimensional Partial Differential Equations Using Deep Learning*. Proceedings of the National Academy of Sciences, **115**(34), 8505–8510.
- [7] Ferguson, R. y Green, A. (2018). *Deeply Learning Derivatives*. arXiv preprint arXiv:1809.02233.
- [8] Becker, S., Cheridito, P. y Jentzen, A. (2019). *Deep Optimal Stopping*. Journal of Machine Learning Research, **20**(74), 1–25.
- [9] Buehler, H., Gonon, L., Teichmann, J. y Wood, B. (2019). *Deep Hedging*. Quantitative Finance, **19**(8), 1271–1291.
- [10] Huge, B. y Savine, A. (2020). *Differential Machine Learning*. arXiv preprint arXiv:2005.02347.
- [11] Ruf, J. y Wang, W. (2020). *Neural Networks for Option Pricing and Hedging: A Literature Review*. Journal of Computational Finance, **24**(1), 1–46.
- [12] Della Corte, P. *et al.* (2023). *Machine Learning for Option Pricing: A Comparative Study of Network Architectures*. Working paper.
- [13] Sridi, A. y Bilokon, P. (2023). *Deep Learning Calibration of Stochastic Volatility Models via Differential Machine Learning*. Working paper.
- [14] Chen, H., Didisheim, A. y Scheidegger, S. (2025). *Deep Surrogates for Finance: With an Application to Option Pricing*. Working paper.